

智能垃圾分类处理系统仿真程序设计

成绩

指导教师：罗志祥

专业班级：光电信息科学与工程 202409

姓名：陈恪瑾

学号：U202413925

联系电话：18571851291

1 任务要求

1.1 实验目的

- 掌握 AT89S52 定时器、中断等外设的基本使用方法；
- 理解智能垃圾桶控制系统的设计逻辑与状态转换过程。

1.2 实验内容

在 Keil 环境中设计一个基于 AT89S52 单片机的智能垃圾分类处理系统。用户通过 P2 口输入、P1 口输出（上电复位初始状态为全 1），模拟小区垃圾回收终端运行状态。系统需满足以下功能：

- 在实验一的基础上修改并扩展设计代码；
- 居民通过按键选择垃圾类型（P2.7、P2.6、P2.5、P2.4 低电平有效，分别对应塑料纸张、玻璃金属、厨余垃圾、危害垃圾）；
- P1.0~P1.3 控制四个垃圾桶盖，高电平为关闭状态，低电平为打开状态；
- 每次开盖持续 8s 后自动关闭，采用定时器中断程序实现延时，计时精度达到 1ms 以内；
- 开盖 8s 期间，P0 口 8 位按剩余时间依次由 1 变至 0，作为开盖进度条显示。

请编写程序，实现用户键入后对应垃圾桶盖的打开与关闭控制。

1.3 提高部分（申优选做）

- 系统初始化后随机生成 4 个 0 ~ 9 的整数，置于寄存器 R0~R3 表示四个垃圾桶剩余容量；每次投递后容量相应减少。投放超出剩余容量后，对应标志位发生变化（自行定义），用于提醒物业清理；再次按键时该桶盖不再打开以避免溢出。

- 自行定义清理键，按下后所有垃圾桶容量恢复为 9，溢出标志位恢复初值。
- 在垃圾桶盖打开的 8s 内，用户可通过按键触发中断（自行设置），停止计时并提前关闭桶盖。
- 其他个性化设计。

2 设计思路

本系统基于 AT89S52 单片机，采用“主循环轮询 + 定时器中断 + 外部中断”的协同控制方式，围绕垃圾分类投放需求划分功能模块，整体设计逻辑如下：

1. **系统初始化模块**：上电后将 P1 置为 0xFF（桶盖全关）、P0 清零、P2 置 0xFF；清空满载标志字节 FULL_FLAGS 并刷新满载指示灯。随后配置 TMOD=11H（T0/T1 均为模式 1），开启 ET0、EX0 和总中断 EA；其中 T0 用于 1ms 节拍计时，INT0（P3.2）用于“提前关盖”外部中断。T1 保持运行，为随机容量生成提供硬件扰动。
2. **主循环按键调度模块**：主循环首先检测清理键 P2.3，按下后将 R0~R3 容量恢复为 9，并清除满载标志；若未清理，则轮询 P2.7~P2.4 四类投放键，检测到低电平后跳转到对应 TRY_BINx 分支执行投放处理。
3. **投放与容量管理模块**：在 TRY_BINx 中先判断对应满载位，满载则拒绝开盖；未满时检查容量寄存器，容量为 0 则立即置满载标志；否则容量减 1，若减到 0 同步置满载并刷新指示灯。通过上述逻辑实现“容量递减、满载封桶、防溢出”的闭环控制。
4. **8 秒开盖窗口与进度显示模块**：OPEN_BIN_8S 子程序接收桶盖位掩码，拉低 P1 对应位开盖，并初始化 SEC_LEFT=8、MS_CNT=03E8H 与 PROGRESS=FFH。开盖期间由 Timer0 每 1ms 中断递减毫秒计数；每累计 1 秒，P0 进度字节右移一位（FF→7F→3F→…→00），SEC_LEFT 递减；当 SEC_LEFT 减至 0 时自动调用 FORCE_CLOSE 关盖。
5. **提前关盖中断模块**：用户在 8 秒窗口内按下提前关盖键（INT0/P3.2 下降沿触发）进入 EXT0_ISR。中断服务程序在 OPENING=1 时立即调用 FORCE_CLOSE，停止 T0 计时并关闭当前桶盖，从而实现“投放完成即中断关盖”的实时响应。

6. **随机数与满载指示模块**：随机子程序读取 TL1/TH1，并经异或、移位与模 10 处理生成 0~9 初值写入 R0~R3。满载指示由 UPDATE_FULL_LED 完成：考虑 P3.2 已被 INT0 占用，程序将四个满载位映射到 P3.0、P3.1、P3.4、P3.5 输出。

3 资源分配

3.1 I/O 口硬件资源分配

I/O 引脚	方向	功能定义	有效电平
P1.0	输出	塑料纸张垃圾桶盖控制	低电平打开，高电平关闭
P1.1	输出	玻璃金属垃圾桶盖控制	低电平打开，高电平关闭
P1.2	输出	厨余垃圾垃圾桶盖控制	低电平打开，高电平关闭
P1.3	输出	危害垃圾桶盖控制	低电平打开，高电平关闭
P2.3	输入	一键清理复位按键	低电平有效
P2.4	输入	危害垃圾投放按键	低电平有效
P2.5	输入	厨余垃圾投放按键	低电平有效
P2.6	输入	玻璃金属投放按键	低电平有效
P2.7	输入	塑料纸张投放按键	低电平有效
P0.0~P0.7	输出	8 秒开盖进度条显示	高位到低位逐步由 1 变 0
P3.0	输出	塑料纸张满载指示信号	高电平点亮
P3.1	输出	玻璃金属满载指示信号	高电平点亮
P3.2/INT0	输入	提前关盖外部中断按键	下降沿触发
P3.4	输出	厨余垃圾满载指示信号	高电平点亮
P3.5	输出	危害垃圾满载指示信号	高电平点亮

3.2 寄存器资源分配

寄存器	功能定义
R0	塑料纸张垃圾桶剩余容量 (0~9)
R1	玻璃金属垃圾桶剩余容量 (0~9)
R2	厨余垃圾桶剩余容量 (0~9)
R3	危害垃圾桶剩余容量 (0~9)
A	指令运算、端口读写、掩码处理与中间数据暂存
TH0, TL0	Timer0 1ms 节拍重装与计时 (FC18H 重装)
TH1, TL1	Timer1 自由运行计数, 作为随机数扰动源

3.3 片内 RAM 资源分配

存储单元	功能定义
20H (FULL_FLAGS)	四个垃圾桶满载标志字节 (bit0~bit3)
21H (TMP_DATA)	LED 刷新与位映射过程的临时缓存
22H (MS_CNT_L)	1 秒计数低字节 (03E8H 低位)
23H (MS_CNT_H)	1 秒计数高字节 (03E8H 高位)
24H (SEC_LEFT)	开盖剩余秒数 (初始化为 8)
25H (PROGRESS)	P0 进度条显示字节 (FFH 右移至 00H)
26H (ACTIVE_MASK)	当前开盖桶的位掩码 (01/02/04/08)
27H (STATE.0=OPENING)	开盖状态位 (1= 正在开盖计时, 0= 关闭)

4 流程图

4.1 主程序流程图

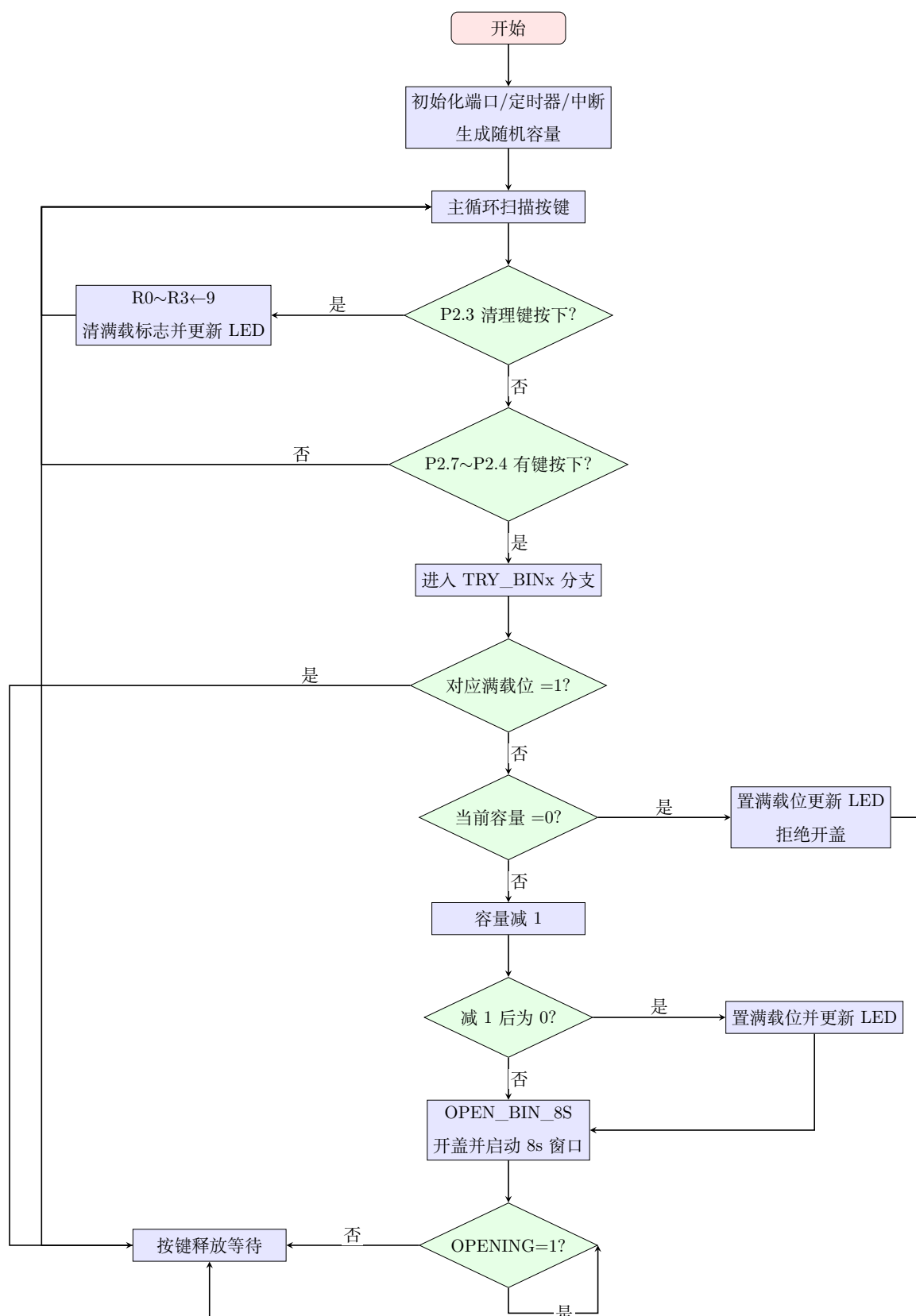


图 1: 主程序流程图

4.2 Timer0 中断服务流程图

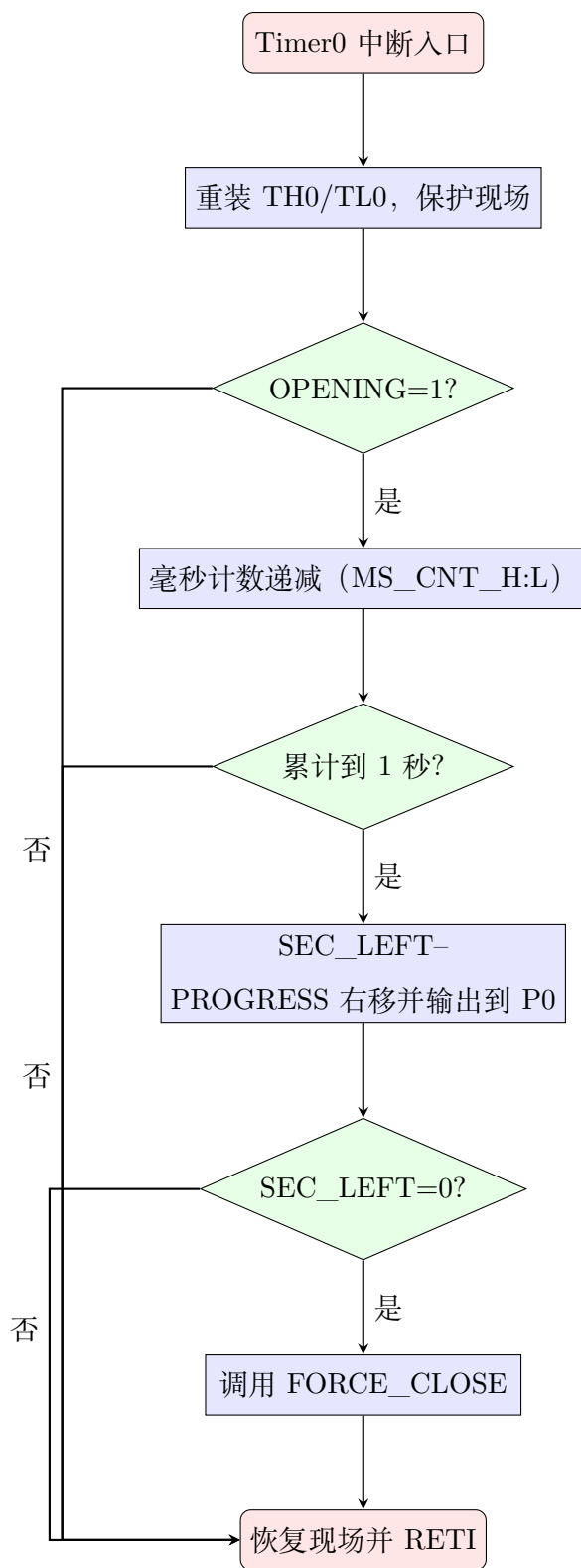


图 2: Timer0 中断服务程序流程图

4.3 INT0 中断服务流程图

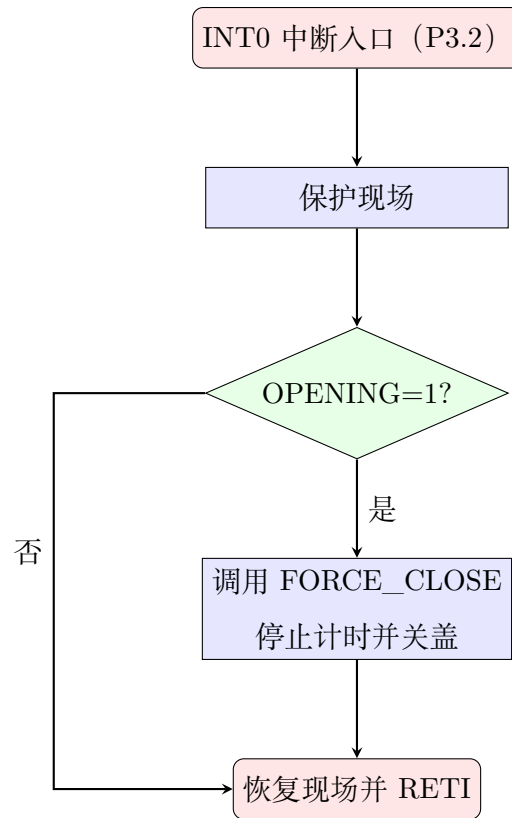


图 3: INT0 提前关盖中断服务流程图

5 源代码（含文件头说明、语句行注释）

; AT89C52(12MHz) 智能垃圾桶控制程序

;

KEY_BINO	BIT	P2.7
KEY_BIN1	BIT	P2.6
KEY_BIN2	BIT	P2.5
KEY_BIN3	BIT	P2.4
KEY_CLEAR	BIT	P2.3
KEY_EARLY	BIT	P3.2

FULL_FLAGS	DATA	020H
TMP_DATA	DATA	021H
MS_CNT_L	DATA	022H

```

MS_CNT_H      DATA    023H
SEC_LEFT      DATA    024H
PROGRESS      DATA    025H
ACTIVE_MASK   DATA    026H
STATE         DATA    027H

```

```

OPENING       BIT      STATE.0

```

; 中断向量

```

ORG          0000H
LJMP        START

```

```

ORG          0003H
LJMP        EXT0_ISR

```

```

ORG          000BH
LJMP        TIMER0_ISR

```

; *setup*函数

```

ORG          0030H

START:

MOV          P1, #0FFH
MOV          P0, #000H
MOV          P2, #0FFH

MOV          FULL_FLAGS, #00H
LCALL        UPDATE_FULL_LED

MOV          TMOD, #11H
LCALL        TO_RELOAD_1MS

CLR          TR0
CLR          TF0
SETB         ETO
SETB         ITO

```

```

SETB    EX0
SETB    EA
SETB    TR1

CLR      OPENING
MOV      ACTIVE_MASK, #00H
MOV      SEC_LEFT, #00H
MOV      PROGRESS, #OFFH

LCALL    RAND_0_9
MOV      R0, A
LCALL    RAND_0_9
MOV      R1, A
LCALL    RAND_0_9
MOV      R2, A
LCALL    RAND_0_9
MOV      R3, A

```

; *loop*函数, 轮询处理按键

MAIN_LOOP:

```

JNB      KEY_CLEAR, DO_CLEAR

JNB      KEY_BIN0, KEY0_HIT
JNB      KEY_BIN1, KEY1_HIT
JNB      KEY_BIN2, KEY2_HIT
JNB      KEY_BIN3, KEY3_HIT
SJMP     MAIN_LOOP

```

KEY0_HIT:

```
LJMP     TRY_BIN0
```

KEY1_HIT:

```
LJMP     TRY_BIN1
```

KEY2_HIT:

```
LJMP     TRY_BIN2
```

KEY3_HIT:

LJMP TRY_BIN3

；清理垃圾桶函数，重置容量和状态

DO_CLEAR:

```
MOV    R0, #09H
MOV    R1, #09H
MOV    R2, #09H
MOV    R3, #09H
MOV    FULL_FLAGS, #00H
LCALL  UPDATE_FULL_LED
LCALL  WAIT_RELEASE
LJMP   MAIN_LOOP
```

；打开垃圾桶函数，判断满载，再尝试打开，其余垃圾桶同理

TRY_BINO:

```
MOV    A, FULL_FLAGS
ANL    A, #01H
JNZ    BINO_DENY

MOV    A, R0
JZ     BINO_MARK_FULL
DEC    R0
MOV    A, R0
JNZ    BINO_OPEN
ORL    FULL_FLAGS, #01H
LCALL  UPDATE_FULL_LED
```

BINO_OPEN:

```
MOV    A, #01H
LCALL  OPEN_BIN_8S
LCALL  WAIT_RELEASE
LJMP   MAIN_LOOP
```

```
BINO_MARK_FULL:
    ORL    FULL_FLAGS, #01H
    LCALL  UPDATE_FULL_LED

BINO_DENY:
    LCALL  WAIT_RELEASE
    LJMP   MAIN_LOOP

TRY_BIN1:
    MOV    A, FULL_FLAGS
    ANL    A, #02H
    JNZ    BIN1_DENY

    MOV    A, R1
    JZ     BIN1_MARK_FULL
    DEC    R1
    MOV    A, R1
    JNZ    BIN1_OPEN
    ORL    FULL_FLAGS, #02H
    LCALL  UPDATE_FULL_LED

BIN1_OPEN:
    MOV    A, #02H
    LCALL  OPEN_BIN_8S
    LCALL  WAIT_RELEASE
    LJMP   MAIN_LOOP

BIN1_MARK_FULL:
    ORL    FULL_FLAGS, #02H
    LCALL  UPDATE_FULL_LED

BIN1_DENY:
    LCALL  WAIT_RELEASE
    LJMP   MAIN_LOOP

TRY_BIN2:
```

```
MOV    A, FULL_FLAGS
ANL    A, #04H
JNZ    BIN2_DENY

MOV    A, R2
JZ     BIN2_MARK_FULL
DEC    R2
MOV    A, R2
JNZ    BIN2_OPEN
ORL    FULL_FLAGS, #04H
LCALL  UPDATE_FULL_LED
```

BIN2_OPEN:

```
MOV    A, #04H
LCALL  OPEN_BIN_8S
LCALL  WAIT_RELEASE
LJMP   MAIN_LOOP
```

BIN2_MARK_FULL:

```
ORL    FULL_FLAGS, #04H
LCALL  UPDATE_FULL_LED
```

BIN2_DENY:

```
LCALL  WAIT_RELEASE
LJMP   MAIN_LOOP
```

TRY_BIN3:

```
MOV    A, FULL_FLAGS
ANL    A, #08H
JNZ    BIN3_DENY

MOV    A, R3
JZ     BIN3_MARK_FULL
DEC    R3
MOV    A, R3
```

```

        JNZ      BIN3_OPEN
        ORL      FULL_FLAGS, #08H
        LCALL   UPDATE_FULL_LED

BIN3_OPEN:
        MOV      A, #08H
        LCALL   OPEN_BIN_8S
        LCALL   WAIT_RELEASE
        LJMP    MAIN_LOOP

BIN3_MARK_FULL:
        ORL      FULL_FLAGS, #08H
        LCALL   UPDATE_FULL_LED

BIN3_DENY:
        LCALL   WAIT_RELEASE
        LJMP    MAIN_LOOP

; 开盖计时器
OPEN_BIN_8S:
        MOV      ACTIVE_MASK, A
        CPL      A
        ANL      P1, A

        MOV      PROGRESS, #0FFH
        MOV      P0, #0FFH
        MOV      SEC_LEFT, #08H
        MOV      MS_CNT_L, #0E8H
        MOV      MS_CNT_H, #03H

        SETB     OPENING
        LCALL   TO_RELOAD_1MS
        CLR      TFO
        SETB     TRO

```

OPEN_WAIT:

```

        JB      OPENING, OPEN_WAIT
        RET

```

; 外部中断0: 提前关盖 (*KEY_EARLY=P3.2*, 低电平触发)

EXT0_ISR:

```

        PUSH    ACC
        PUSH    PSW
        JNB     OPENING, EXT0_ISR_EXIT
        LCALL   FORCE_CLOSE

```

EXT0_ISR_EXIT:

```

        POP     PSW
        POP     ACC
        RETI

```

; 1ms延时中断

TIMERO_ISR:

```

        PUSH    ACC
        PUSH    PSW

        LCALL   TO_RELOAD_1MS
        JNB     OPENING, TO_ISR_EXIT

        MOV     A, MS_CNT_L
        JNZ     MS_DEC_LOW

        MOV     A, MS_CNT_H
        JZ      ONE_SECOND_ELAPSED
        DEC     MS_CNT_H
        MOV     MS_CNT_L, #OFFH
        SJMP    TO_ISR_EXIT

```

MS_DEC_LOW:

```

        DEC     MS_CNT_L

```

```

MOV    A, MS_CNT_L
JNZ    TO_ISR_EXIT
MOV    A, MS_CNT_H
JNZ    TO_ISR_EXIT

```

ONE_SECOND_ELAPSED:

```

MOV    MS_CNT_L, #0E8H
MOV    MS_CNT_H, #03H

DEC    SEC_LEFT
CLR    C
MOV    A, PROGRESS
RRC    A
MOV    PROGRESS, A
MOV    P0, A

MOV    A, SEC_LEFT
JNZ    TO_ISR_EXIT
LCALL  FORCE_CLOSE

```

TO_ISR_EXIT:

```

POP    PSW
POP    ACC
RETI

```

; 强制关盖

FORCE_CLOSE:

```

CLR    TRO
MOV    A, ACTIVE_MASK
ORL    P1, A
MOV    ACTIVE_MASK, #00H
CLR    OPENING
MOV    P0, #000H
RET

```

TO_RELOAD_1MS:

; 12MHz 下定时器计数周期 $1\mu s$, $1ms$ 需 1000 次计数

; 重装值 = $65536 - 1000 = FC18H$

MOV TH0, #0FCH

MOV TL0, #018H

RET

; 随机数生成器

RAND_0_9:

MOV A, TL1

XRL A, TH1

XRL A, #05AH

RL A

XRL A, #01DH

RR A

ANL A, #01FH

RAND_MOD10:

CLR C

SUBB A, #0AH

JNC RAND_MOD10

ADD A, #0AH

RET

UPDATE_FULL_LED:

; 释放 P3.2 作为 INT0 键输入, 满载灯改用 P3.0/P3.1/P3.4/P3.5

MOV A, FULL_FLAGS

ANL A, #03H

MOV TMP_DATA, A

MOV A, FULL_FLAGS

ANL A, #0CH

```

RL      A
RL      A
ORL     A, TMP_DATA
MOV     TMP_DATA, A

MOV     A, P3
ANL     A, #0CCH
ORL     A, TMP_DATA
MOV     P3, A
RET

```

WAIT_RELEASE:

```

MOV     A, P2
ANL     A, #0F8H
CJNE    A, #0F8H, WAIT_RELEASE
RET

END

```

6 程序测试方法与结果

本节采用 Keil 仿真环境进行功能验证。测试遵循“按键输入-端口状态-寄存器变量-中断响应”四类观测指标，重点验证主程序与中断程序的协同正确性。

6.1 测试方法

1. **端口观测**：观察 P1（桶盖控制）、P0（8 位进度条）、P3（满载指示）电平变化。
2. **变量观测**：观察 R0~R3 容量、FULL_FLAGS、SEC_LEFT、OPENING 等关键变量。
3. **中断观测**：分别触发 Timer0 周期中断与 INT0 外部中断，检查是否按预期执行关盖流程。

6.2 测试结果

(1) 上电初始化与随机容量测试

操作：系统复位后进入主程序初始化。

预期：P1=FFH（桶盖全关），P0=00H，FULL_FLAGS 清零，R0~R3 生成 0~9 随机容量。

结果：与预期一致。

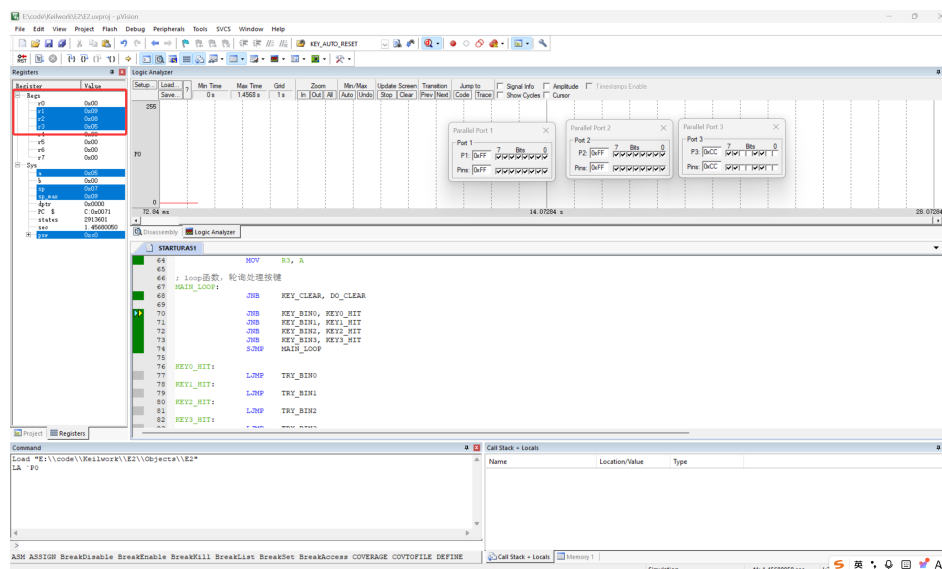


图 4: 初始化与随机容量测试结果

(2) 分类按键开盖功能测试

操作：以 P2.7 为例，将其拉低模拟“塑料纸张”投递按键。

预期：P1.0 拉低开盖，进入 8s 开盖窗口；若该桶未满载则容量减 1。

结果：与预期一致。

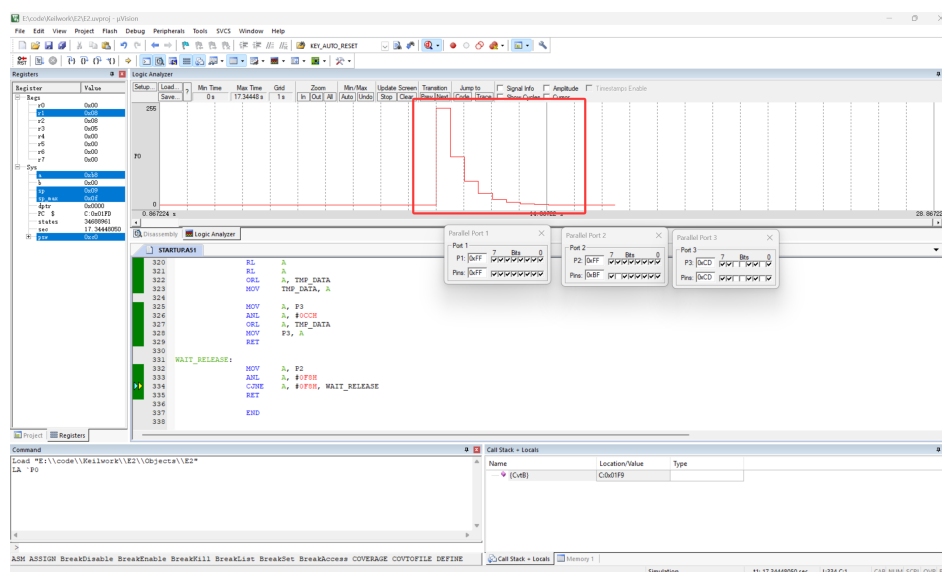


图 5: 分类按键开盖测试结果

(3) 8 秒自动关盖与 P0 进度条测试

操作：触发任一垃圾桶开盖后，持续观察 Timer0 中断过程。

预期：每 1 秒 SEC_LEFT 减 1，同时 P0 由 FFH 逐步右移至 00H；8 秒到期后自动调用 FORCE_CLOSE，P1 对应位恢复高电平。

结果：与预期一致。

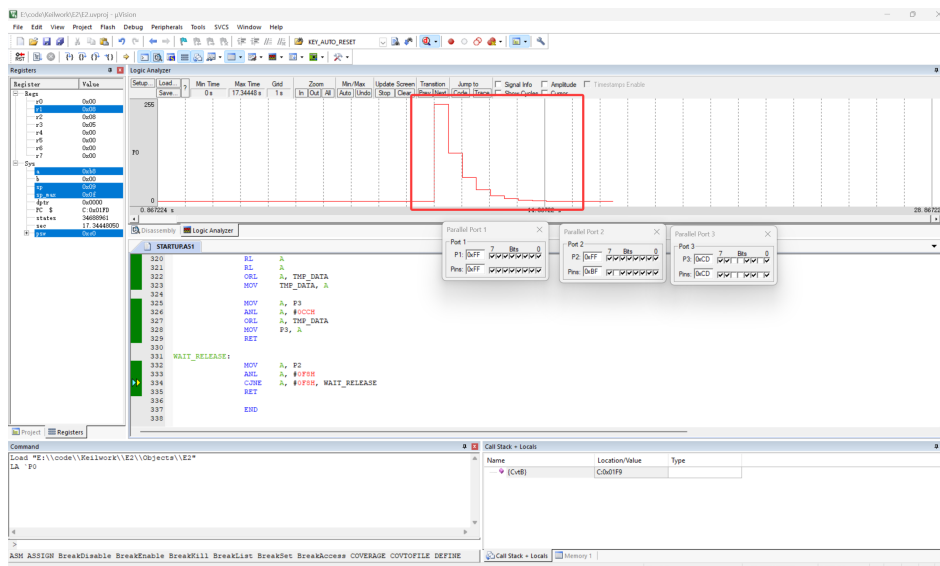


图 6: 自动关盖与进度条测试结果

(4) 满载封桶与指示灯测试

操作：连续投递使某一桶容量减至 0，并再次触发该桶按钮。

预期：对应满载标志位置 1，后续按钮不再开盖；满载指示口 (P3.0/P3.1/P3.4/P3.5) 对应位置高电平。

结果：与预期一致。

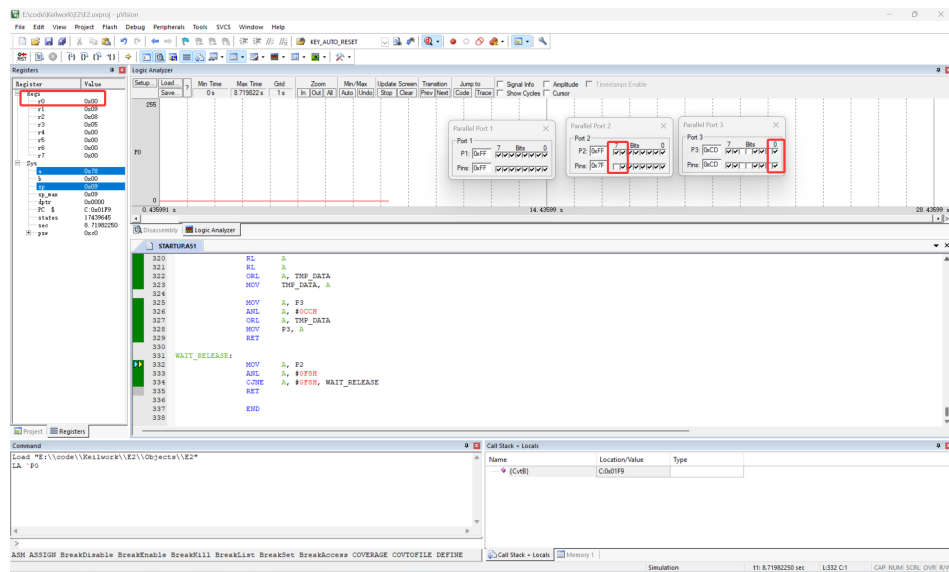


图 7: 满载封桶测试结果

(5) 清理键复位测试

操作：按下 P2.3 清理键。

预期：R0~R3 全部恢复为 9，FULL_FLAGS 清零，满载指示熄灭。

结果：与预期一致。

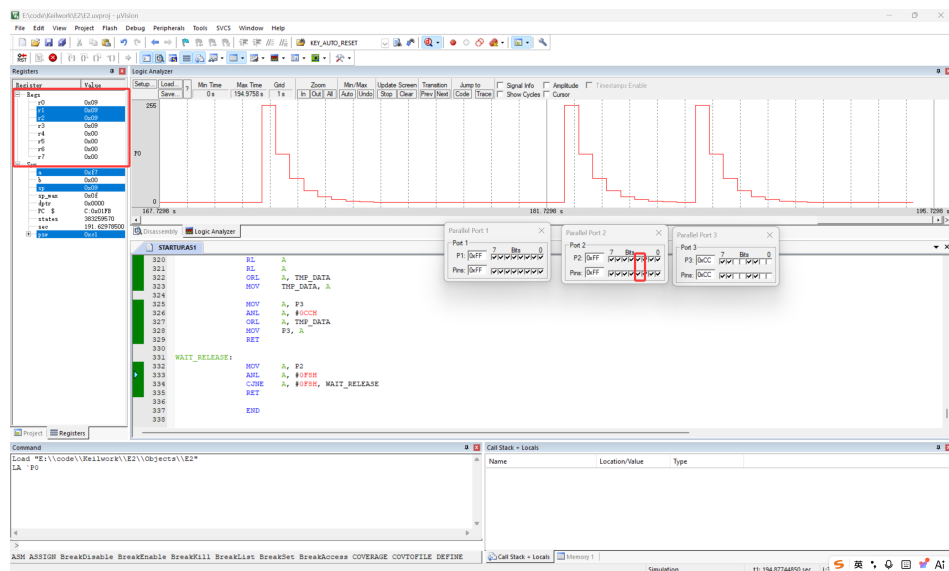


图 8: 清理复位测试结果

(6) INT0 提前关盖中断测试

操作：开盖窗口内触发 P3.2 (INT0) 下降沿中断。

预期：进入 EXT0_ISR，立即调用 FORCE_CLOSE，停止 T0 计时并提前关闭当前桶

盖，OPENING 清零。

结果：与预期一致。

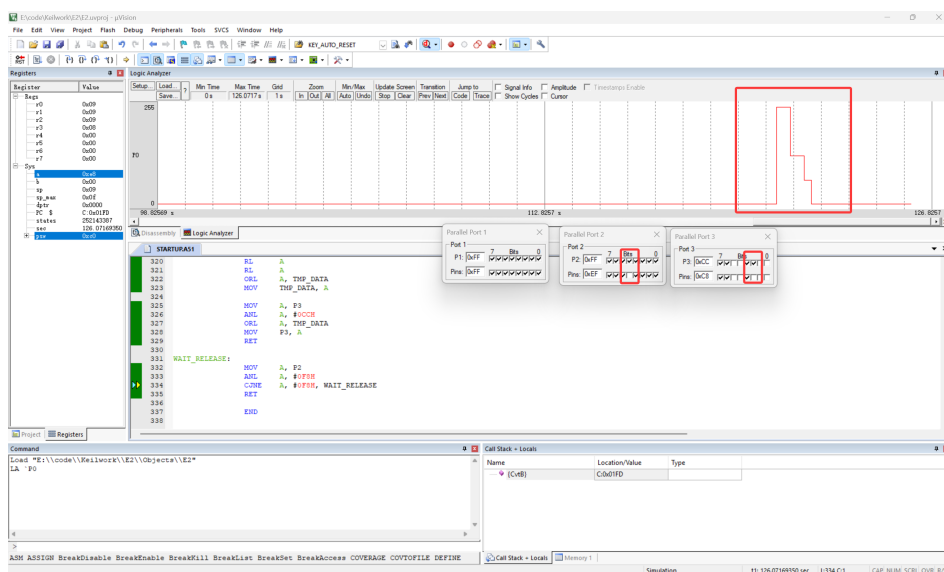


图 9: 提前关盖中断测试结果

综上，程序已完成任务书要求的全部核心功能，并实现了容量管理、清理复位和中断提前关盖功能。

7 实验问题分析与对策

1. 定时器 1ms 节拍精度问题：

- **问题：**实验要求通过定时器中断实现 8s 开盖窗口，但在仿真的配置中误将晶振频率设置为 24MHz，导致关盖时间只有 4s，不符合设计要求。
- **对策：**将晶振频率调整为 12MHz，确保代码逻辑与硬件外设保持一致，从而达到实验需求。

2. 提前关盖中断与端口复用冲突问题：

- **问题：**提前关盖采用 INT0 时必须占用 P3.2，而原方案将 P3.0~P3.3 全部用于满载指示，导致引脚功能冲突。
- **对策：**将 P3.2 专用于 INT0 下降沿触发，满载指示调整为 P3.0、P3.1、P3.4、P3.5。并在 UPDATE_FULL_LED 中完成位映射，确保中断输入与指示输出互不干扰。

3. 按键连发与释放判定问题：

- **问题：**低电平按键在未释放时会被主循环重复识别，造成一次按压触发多次投递分支。
- **对策：**所有投递分支和清理分支结束前统一调用 WAIT_RELEASE，等待相关按键恢复高电平后再返回主循环，抑制连发，提高交互稳定性。

本人承诺: 本报告内容真实，无伪造数据，无抄袭他人成果。本人完全了解学校相关规定，如若违反，愿意承担其后果。

签字：

陈恪瑾

2026 年 4 月 26 日